

The Block Cipher: GRAND CRU

Johan Borst

K.U. Leuven
Dept. Elektrotechniek-ESAT/COSIC
Kardinaal Mercierlaan 94
B-3001 Heverlee België
Johan.Borst@esat.kuleuven.ac.be

Abstract

This document contains a description of the block cipher GRAND CRU, which supports a 128-bit block size and 128-bit key size. Its security is based on the strategy of multiple layered security which also is described in this article. This strategy is aimed at combining different security motivations to enhance security. The design of GRAND CRU is aimed at 8 bit architectures, like low end smart cards. The cipher is largely based on Rijndael.

Contents

1	Introduction	3
2	Design Rationale	4
2.1	Motivation	4
2.2	Strategy	5
2.3	A Proposal	8
2.3.1	Design Goals	8
2.3.2	Rijndael	9
2.3.3	Security Motivations	13
2.3.4	Keyed Permutations	13
2.3.5	Keyed Byte-Wise Rotations	15
2.3.6	Shannon's Construction	15
2.3.7	The Cipher	17
2.3.8	The One-Way Functions	17
2.3.9	Round Key Derivation	18
3	Specification	20
3.1	Notation	20
3.2	Transformations	20
3.3	The Cipher	23
3.4	Key Schedule	23
4	Implementation	26
4.1	Efficiency	26
4.2	Avoiding Implementation Weaknesses	27
5	Security Analysis	29
5.1	The Permutation Cipher	29
5.2	The Rotation Cipher	30
5.3	The Outer Round Cipher	30
6	Advantages and Limitations	31
7	Miscellaneous	32
7.1	Intellectual Property	32
	Bibliography	33
A	S-Box	35

1 Introduction

The block cipher GRAND CRU has a 128-bit block size and supports keys of size 128 bits. Its security is based on the strategy of multiple layered security which also is described in this document. This strategy is aimed at combining different security motivations to enhance security. The design of GRAND CRU is primarily aimed at 8 bit architectures, like low end smart cards, although for most applications its performance is sufficient. The basic design of the cipher is largely based on Rijndael.

The strategy of multiple layered security is a new block cipher design strategy. The idea behind this strategy is to have a structure that contains several different strong block ciphers. Each cipher uses a set of different subkeys. The knowledge of one set of subkeys does not reveal any (useful) information about the other key sets. Hence, an attacker would be forced to use different attack methods. This strategy can be used to divide possible weaknesses over different components. Most important benefit of this is that this strategy can be used to enhance security against side-channel attacks. GRAND CRU has been designed according to this strategy.

The remainder of this document is organized as follows.

- Sect. 2 contains an elaborate design rationale of GRAND CRU. Due to the unconventional although straight forward design strategy we preferred to start this document by giving a description of the whole design process. We also think that an understanding of the design strategy contributes to a transparent view on the cipher. *Readers that want to have quickly a complete (but dry) description of the cipher can skip this section and go directly to Sect. 3.*
- Sect. 3 contains a complete description of GRAND CRU. In this aspect it summarizes Sect. 2.
- Sect. 4 contains implementation related issues, including estimated computational efficiency in software and guidelines for implementers to avoid implementation weaknesses.
- Security aspects are studied in Sect. 5.
- The advantages and limitations of the cipher are addressed in Sect. 6.

2 Design Rationale

This section contains the design rationale of GRAND CRU and is organized as follows. In Sect. 2.1 we motivate the design strategy and introduce the concept of multiple layered security. In Sect. 2.2 the strategy is formalized. A concrete proposal of the design of a block cipher based on Rijndael is made in Sect. 2.3.

2.1 Motivation

In the last few years block cipher design has received increased attention. This was caused by the initiative from NIST to replace DES by AES as an American federal standard [1]. Fifteen algorithms were accepted and after a first selection process there remained five possible candidates: MARS, RC6TM, Rijndael, Twofish and Serpent.

The AES predecessor DES has survived more than 20 years of cryptanalysis and a conservative approach would have been to design the new AES according to similar principles, bearing in mind the progress in cryptanalytic research. However, DES was designed by IBM and NSA and its design criteria were never completely published. This has made the AES selection process an interesting stage for the evaluation of different design strategies. As this process is nearing its end it is interesting to look back at the selection process and arguments, that lead to the remaining candidates. One can say about the five remaining candidates that they are sufficiently efficient on many platforms and sufficiently secure against currently known attacks. The same can actually be said about several other ciphers from the group of original proposals. For example CAST-256, (the tweaked version of) CRYPTON, DFC, E2 and SAFER+ would be sufficiently secure and sufficiently efficient for many applications on most platforms. On the other hand, except for resistance against exhaustive key search, few things can (and have been) be said about possible attacks that are the result of evolving cryptanalytic research. However, one statement can be made, which we use as a working assumption.

Assumption 2.1 *The probability that one of a group of x (for example 5 ciphers, as in the case of 5 AES finalists) ciphers gets broken in the future is bigger than the probability that all x would get broken.*

The notice of this assumption even has had an impact on the developments of the AES selection. Although the original purpose was to use one standard, soon comments were addressed to NIST about choosing multiple standards. At the third AES conference there was even a forum dedicated to this issue. We use Assumption 2.1 as the basis of a new design strategy: block cipher design with *multiple layered security*.

If one has a set of x unbroken (but not provably secure) ciphers, one can choose one and use it for encryption. However, there is always the possibility that the chosen cipher gets broken and a lot of encrypted information gets compromised. To decrease the probability of compromise of the encrypted information, one could use a chain of the x secure ciphers with independent keys.

However, this reduces the performance by a factor of about x and would also result in the need for about x times more hardware resources. In this paper we suggest a similar approach from the security point of view, that still gives opportunity for efficient cipher design. *In particular it gives rise to much more efficient designs than compared to the use of a sequence of several ciphers.*

The basic idea is to fit x secure ciphers (*subciphers*) into a similar design, such that if $x - 1$ cipher keys are known still a secure cipher would remain. We call this notion of security *multiple layered security*. The user key would either have to be sufficiently long to contain the user key of each subcipher or the keys of the subciphers would need to be derived from the user key with one or several one-way functions, such that these keys would not be distinguishable from independently chosen keys. For the cipher to be implementable on platforms where few memory for code is available, such as smart cards it would be preferable that the one-way function(s) use the same structure as the cipher.

Another feature of this design strategy is that it can be used to design block ciphers that are particularly suitable for implementations that resist side-channel attacks on a wide variety of platforms. In order to achieve this the subciphers should have different operations for involving the keys in the computation, since these are the operations whose information leakage via most practical side-channels is exploited.

In this chapter we formalize this design approach, discuss the different design strategies that could be combined and propose a design, based on Rijndael.

2.2 Strategy

The concept of creating a more secure block cipher by chaining x secure ciphers is well known [24]. The most important idea behind it is that if $x - 1$ ciphers would get broken, one still would be left with a secure cipher. Another idea is that in this way one can create ciphers with larger key length, without having to redesign the key schedule. The most prominent example of this is triple-DES. The general acceptance of triple-DES as replacement for DES in applications where a 56-bit key does not suffice illustrates the confidence in this method. The main disadvantage of this strategy is the performance drop, i.e., a chain of x ciphers is approximately x times as slow as one cipher.

One does not have to chain x ciphers to combine them such that one would be left with a secure cipher if the keys of $x - 1$ would be known. An example of this is XDES, which was proposed by Rivest to strengthen DES against exhaustive key search. It is described and analyzed by Kilian and Rogaway in [15]. The cipher is a DES, where 64-bit keys are xored to the input and output:

$$\text{XDES}_{K_1||K_2||K_3}(X) = \text{DES}_{K_1}(X \oplus K_2) \oplus K_3. \quad (1)$$

One can see this as an interleaved combination of DES and a cipher consisting out of two outer key mixings with a (known) confusion layer in between. The latter was already proposed by Shannon in [24]. Based on the method of outer key mixing there are some attacks possible; in the case of XDES see the analysis by Kilian and Rogaway in [15] and the attacks by Biryukov and Wagner in [6].

Another related strategy is the design strategy of the family of COCONUT ciphers as described by Vaudenay in [25]. A COCONUT cipher is a chain of ciphers $E_1(E_2(E_3))$, where E_2 (and hence the whole cipher) is perfectly decorrelated to the order two, hence provably secure against a large class of attacks.

The case of XDES is interesting since the keyed DES in the middle would represent an unkeyed confusion and diffusion layer, if its key was known. This leads to the following:

Design strategy

1. Start with a secure block cipher (with well-defined designing strategy and analysis).
2. Change unkeyed operations in that cipher into keyed operations or add (keyed) operations such that:
 - (a) The security of the original cipher is not compromised, i.e., the design principles of the original cipher and the motivations or proofs for security against certain attacks should still hold for the remaining cipher and should still hold if the keys of the extra operations are known. Note that in this case there can be different attack scenarios: *known subcipher key attacks* and *chosen subcipher key attacks*. The latter is only possible if the subcipher keys are independent (or can not be distinguished from independent keys).
 - (b) One can divide the keys of the keyed operations into $x - 1$ sets. Together with the subkeys of the original cipher one has x sets. In this way x ciphers are defined. Ideally if any $x - 1$ sets were known, then the remaining cipher should still be secure (note that this implies the item 2.(a)). We say that the cipher provides *multiple layered security*. But if this condition is not met optimally, the designed cipher still can be useful. Again one can distinguish known and chosen subcipher key attacks.
 - (c) There are no non-trivial alternative representations of the cipher in less than x subciphers.
3. Design efficient and secure key schedules for the $x - 1$ ciphers that had none.
4. The cipher is keyed by a set of keys $K_0, \dots, K_{x-1}$, where
 - this set forms the user key; or
 - this set is derived from the user key with one or several one-way functions. This derivation should be done in such a way that if $x - 1$ derived keys are known, the user key should not be deducible.
5. The remaining cipher is still (sufficiently) efficient. In particular it should be at least as efficient as a sequence of x ciphers that provide approximately the same level of security.

6. The remaining cipher is secure against ad-hoc analysis.

The subciphers ideally complement each other from a security point of view. It is not wanted that a successful attack on one subcipher implies a successful attack on another subcipher. This means:

In a classical cryptanalytic sense The security of the different subciphers should rely on different strategies. For example, one does not want two subciphers, for which the security against linear [20] and differential cryptanalysis [3] is mainly based on the properties of data-dependent rotations.

With respect to side-channel analysis With this strategy one can increase resistance against side-channel attacks. This is caused by the fact that all practical side-channel attacks exploit side-channel information that is leaked by key dependent operations. Hence, it is important to implement or choose the key dependent operations such that they do not leak side-channel information. For the other operations this is of less importance: for example SPA is possible because all operations leak information through their power consumption, but it does not have to be exploitable. Different operations can give rise to exploitation by different side-channels. For example, the xors with keys in DES can be vulnerable to power analysis, while the data dependent rotations in RC5 for some implementations can leak information through timing analysis as shown by Handschuh and Heys in [14].

If necessary the operations that leak side-channel information can usually be implemented in a way such that there is no exploitable leakage. The method to do this depends on the platform. Depending on the platform some operations might be more suitable for resistant implementation against their side-channel weaknesses than others.

Hence, there are several advantages in choosing the key dependent operations in the different subciphers different from each other. Firstly, the resistance of the against side-channel attacks is guaranteed by the subcipher whose potential vulnerability can be most efficiently countered on the specific platform. Secondly, using several layers of different key dependent operations gives a higher probability against side-channel attacks that might evolve in the future. Hence, this would make it more improbable that a situation would occur where smart card industry is forced to do a lot of investment due to a discovery like power analysis.

Instead of choosing different key dependent operations one might allow key mixing with the same operations but use different code (or hardware) to implement these operations.

The above described design strategy is very conservative and intuitively comprehensible. However, to apply it in practice several hurdles have to be taken. Suppose we should choose DES in step 1. Since the design principles of DES are not known, there is little motivation to change or add operations to the internals. Hence, it is not possible to do much more than an XDES-like construction.

Fortunately, not in the last place caused by the AES selection process, there are several alternative block ciphers, that have received much analysis and that are designed (or at least analyzed) with a clear design strategy. A little unfortunate, but not unexpected, is that none of these ciphers can be efficiently combined with each other. So a construction with one or more subciphers that is less analyzed will probably result.

Efficiency The items of the design strategy mainly concern the security aspects of the design. Next we list some miscellaneous items that concern the efficiency of the design.

- The addition of extra operations should be minimized, since every extra operation decreases the performance.
- In many applications it would be beneficial if the extra-designed key derivation procedures and the one-way functions would use the same structure of operations as used in the algorithm. This would reduce the code size in applications with memory restrictions and the implementation cost for hardware applications.
- If in the application also a secure one-way function is implemented such as SHA-1, SHA-2 (currently in development) or RIPEMD-160, then these can be used for subcipher key derivation if there are no performance restrictions.
- The use of several subciphers leads to an increase of the amount of subkeys and hence will typically require more memory. The comparative high memory requirements seem to be inherent to this design strategy.

In the next section we will describe a combination of 4 subciphers based on the AES candidate Rijndael.

2.3 A Proposal

In this section we use the design strategy to construct a four-layered cipher, based on Rijndael. Hence, step 1 in the design strategy is to choose Rijndael.

2.3.1 Design Goals

We explicitly state that the following principle, in which our design strategy differs from conventional design. *The efficiency of the design should be compared to ciphers that offer an equivalent level of security, i.e., multiple layered security. In our case: sequences of ciphers, that have different security motivations. With efficiency we mean: performance, code size, gate count, etc.* Concerning security and efficiency the design should have the following properties.

- The block size of the cipher is 128 bits: the nominal choice to resist dictionary like attacks and attacks based on the birthday paradox.

- It can be used with a 128-bit user key, which is the minimal specified key length. Alternative key sizes (higher than 128 bits) can be defined.
- Each subcipher has a 128-bit key. These keys are derived from the user key using a one-way function. If the user key size is greater than $4 \cdot 128 = 512$ bits the one-way function does not have to be applied.
- The cipher is sufficiently resistant against all known short-cut attacks.
- Each subcipher is sufficiently resistant against all known short-cut attacks.
- The design should be efficient on platforms that are most vulnerable to side-channel analysis. The most suitable targets for this are smart cards. These mainly use 8-bit processors. Hence, for efficiency we keep the operations as much byte-oriented as possible.
- The code of the implementation of the cipher should fit in the memory a typical smart card.
- For software implementation on other platforms the efficiency should be acceptable. In particular we do not emphasize possible optimization on general purpose 32-bit (or 64-bit) architectures since on these platforms, the law of Moore seems to be more applicable than on low end smart cards.
- The security goals of our design strategy also can be achieved by using several ciphers in sequence. Hence, a four-layered cipher should be more efficient as a sequence of 4 ciphers. We put our goal further: the cipher should be more efficient (taking into account performance and memory requirements) than any three of the five remaining AES candidates in sequence.

2.3.2 Rijndael

Rijndael, designed by Daemen and Rijmen [10], is currently one of the five remaining AES candidates. Its design is based on the Wide Trail Strategy as described in [9]. Rijndael consists of four transformations. All transformations operate on 8-byte strings, that we call *states*, and can be visualized in different ways. The layers can be described as follows. We use the notations for the transformations as given by equations (34), (35), (36) and (37).

Round Key Addition σ This is performed by xoring the input with a 128-bit round key K_i :

$$\sigma(A)_{K_i} = A \oplus K_i, \text{ for } K_i, A \in \{0, 1, \dots\}^{128}. \quad (2)$$

Non-linear Substitution Layer γ This is called the ByteSub transformation in [10]. It uses an 8×8 S-box S , which operates on each of the bytes of the state independently:

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\gamma} \begin{pmatrix} S(x_{03}) & S(x_{02}) & S(x_{01}) & S(x_{00}) \\ S(x_{13}) & S(x_{12}) & S(x_{11}) & S(x_{10}) \\ S(x_{23}) & S(x_{22}) & S(x_{21}) & S(x_{20}) \\ S(x_{33}) & S(x_{32}) & S(x_{31}) & S(x_{30}) \end{pmatrix}. \quad (3)$$

The S-box is constructed such that it has optimal properties against LC and DC. This is summarized by the following property.

Property 2.1 *With respect to linear and differential cryptanalysis, the Rijndael S-box has the following properties.*

- The linear approximation with maximal bias for S has bias 2^{-3} .
- The differential with maximal probability for S has probability 2^{-6} .

Furthermore the S-box is constructed such that its properties are hard to exploit in algebraic attacks like interpolation attacks.

Diffusion Layers π and θ The transformation π or ShiftRow transformation is a permutation of the bytes of a state. It operates on the rows of a state. It rotates the second row one, the third row two, and the fourth row three bytes. Hence, $\pi : \{0, 1\}^{128}$ is defined as

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\pi} \begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{10} & x_{13} & x_{12} & x_{11} \\ x_{21} & x_{20} & x_{23} & x_{22} \\ x_{32} & x_{31} & x_{30} & x_{33} \end{pmatrix}. \quad (4)$$

For security its most important property is:

Property 2.2 *For π the four rotation amounts of the rows are different.*

This property ensures (in the iterated round structure) resistance against attacks using truncated differentials and resistance against the dedicated Square attack.

The transformation θ or MixColumn transformation operates separately on each column of a state:

$$(x_3|x_2|x_1|x_0) \xrightarrow{\theta} (\theta(x_3)|\theta(x_2)|\theta(x_1)|\theta(x_0)). \quad (5)$$

The columns of a state are considered as polynomials over $\text{GF}(2^8)$ and multiplied modulo $x^4 + 1$ with a fixed polynomial $c(x)$. This can be represented as the following matrix multiplication:

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\theta} \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \cdot \begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix}. \quad (6)$$

This transformation has a very important role with respect to the wide trail strategy. It provides diffusion in the sense that it guarantees a high number of active S-boxes. For this the following property is important.

Property 2.3 *For each $i = 0, \dots, 3$ the following holds that θ_i has branch number 5. This means:*

- *Each pair of corresponding four tuples of input bytes and output bytes of θ_i contains at least 5 non-zero bytes or are all zero.*
- *Since θ_i is linear also for the difference of each two pairs of corresponding four tuples of input bytes and output bytes of θ_i it holds that at least 5 bytes are non-zero or all bytes are zero.*

This property ensures that a linear approximation or a differential of one round always involves at least five active S-boxes.

Round Transformation One round ρ of Rijndael can be defined as

$$\rho_K = \theta \circ \pi \circ \gamma \circ \sigma_K. \quad (7)$$

There is a different final round ω :

$$\omega_{K_9, K_{10}} = \sigma_{K_{10}} \circ \pi \circ \gamma \circ \sigma_{K_9}. \quad (8)$$

A Rijndael encryption with a 128-bit key consists of 9 rounds ρ_{K_i} followed by one round $\omega_{K_9, K_{10}}$, where $K_0^0, \dots, K_{10}^0$ are 128-bit round keys derived from the user key K^0 :

$$\text{Rijndael}_{K^0} = \sigma_{K_{10}^0} \circ \pi \circ \gamma \circ \sigma_{K_9} \circ \prod_{i=8}^0 (\theta \circ \pi \circ \gamma \circ \sigma_{K_i^0}), \quad (9)$$

where $\prod$ stands for concatenation of transformations. Rijndael decryption has the same structure It can be given by

$$\text{Rijndael}_{K^0}^{-1} = \sigma_{K_0^0} \circ \pi^{-1} \circ \gamma^{-1} \circ \sigma_{K_1} \circ \prod_{i=2}^{10} (\theta \circ \pi^{-1} \circ \gamma^{-1} \circ \sigma_{K_i^0}), \quad (10)$$

Key Schedule The key schedule is defined by a transformation $f_{\text{rk}} : \{0, 1\}^k \rightarrow \{0, 1\}^k$ that derives as much k -bit strings as necessary to form all subkeys. Each k -bit string is derived by applying ψ on the previous one. This procedure has the advantage that during encryption with Rijndael only one k -bit string has to be stored for key material; the needed key subkeys can be efficiently computed *on-the-fly*. For a 128-bit key the first subkey is the userkey and the i th subkey is derived from the $i - 1$ th subkey.

We describe the key schedule for a userkey of 128 bits. For a description for other key lengths we refer to [10]. Similar as for the cipher states such 128-bit subkey can also be represented by a 4×4 -array:

$$k_{33}k_{32}k_{31}k_{30}k_{23} \dots k_{01}k_{00} = \begin{pmatrix} k_{03} & k_{02} & k_{01} & k_{00} \\ k_{13} & k_{12} & k_{11} & k_{10} \\ k_{23} & k_{22} & k_{21} & k_{20} \\ k_{33} & k_{32} & k_{31} & k_{30} \end{pmatrix} = (k_3|k_2|k_1|k_0). \quad (11)$$

We define the transformation Rotl, which operates on columns of a key state. It rotates the bytes of a column as follows.

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \xrightarrow{\text{Rotl}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_0 \end{pmatrix} \quad (12)$$

Furthermore the transformation Γ or SubByte also operates on the bytes of a column and is defined as:

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \xrightarrow{\Gamma} \begin{pmatrix} S(x_0) \\ S(x_1) \\ S(x_2) \\ S(x_3) \end{pmatrix}. \quad (13)$$

Rijndael with a 128-bit key length requires 11 128-bit subkeys that we denote with

$$K_{10} \dots K_1, K_0 = (K'_{43}|K'_{42}|K'_{41}|K'_{40}), \dots, (K'_7|K'_6|K'_5|K'_4), (K'_3|K'_2|K'_1|K'_0). \quad (14)$$

If the userkey is denoted by K^u then

$$K_0 = K^u. \quad (15)$$

The remaining key material is derived by

$$K'_i = K'_{i-4} \oplus \Gamma(\text{Rotl}(K'_{i-1}) \oplus c(i/4)), \text{ if } 4|i, \quad (16)$$

$$K'_i = K'_{i-4} \oplus K'_{i-1} \text{ else,} \quad (17)$$

where $c(i)$ are the round constants of the key schedule as defined in Table 1. In the following we shall use a shortened notation to denote an i th subkey that is derived from user key K^u :

$$K_i = f_{\text{rk}}(K^u, i). \quad (18)$$

The key schedule has the following property which is of importance in applications where the memory available for subkeys is restricted.

Property 2.4 *The Rijndael key schedule permits on-the-fly subkey computation. In each round only a 128-bit subkey needs to be stored. The next subkey can be computed from the previous one.*

Table 1: Round constants of Rijndael key schedule, which are defined for up to a possible number of 30 rounds.

i	$c(i)$	i	$c(i)$	i	$c(i)$
1	0x01	11	0x6c	21	0x97
2	0x02	12	0xd8	22	0x35
3	0x04	13	0xab	23	0x6a
4	0x08	14	0x4d	24	0xd4
5	0x10	15	0x9a	25	0xb3
6	0x20	16	0x2f	26	0x7d
7	0x40	17	0x5e	27	0xfa
8	0x80	18	0xbc	28	0xef
9	0x1b	19	0x63	29	0xc5
10	0x36	20	0xc6	30	0x91

2.3.3 Security Motivations

The formal motivations for the security of Rijndael are based on the properties 2.1, 2.2, and 2.3. Combined with the structure of Rijndael, i.e., the specified sequence of transformations, they provide motivations for its resistance against linear and differential cryptanalysis and similar attacks, as well as more dedicated structural attacks like interpolation attacks and attacks using truncated differentials, which would exploit the byte oriented design.

Step 2 in the design strategy is change unkeyed permutations into keyed operations and/or add keyed operations without violating the original security motivations, which here are the properties 2.1, 2.2, and 2.3. We make and motivate the changes in the following sections.

2.3.4 Keyed Permutations

This security motivations still hold when we make the following changes.

- Replace π with a keyed byte permutation π_K^1 . This is a permutation that rotates three rows of the state with respectively one, two or three bytes. Which row is rotated by which amount is determined by the value of K . This key can take on $4! = 24$ different possible values. The permutation π_K^1 has the same property as given by property 2.2: all the rotation amounts of the rows are different. It does not interfere with the other security motivations.
- Add before each permutation π_K^1 a keyed byte permutation π_K^2 , that operates column-wise on a state and permutes the bytes of each column, depending on the value of K . This key can take on $(4!)^4$ possible values. This addition does not interfere with property 2.1 since it only permutes bytes. It does not interfere with property 2.3: rearranging the bytes in a

column does not change the branch number, i.e., the number of non-zero bytes stays the same; they only might be at a different position. Finally it does not interfere with property 2.2. In the equation representing the cipher each π_K^2 is the neighbour of a θ or can be shifted to be one, without altering the outcome of the cipher. Hence, one can view this addition as changing θ for $\theta \circ \pi_K^2$, which does not interfere with π_K^2 or its (relevant) properties.

Both permutations can be combined in one keyed permutation π_K , where K can take on $(4!)^5 \approx 2^{22}$ possible values. We propose to replace π with π_K . As motivated, this can be done without compromising the security arguments of Rijndael.

The permutation can be implemented by representing each of the five separate permutations on four bytes or rows by 5 key bits that represent a value smaller than 24. A table of 24 entries (that could have a length of 6 bits, but 8 bits seems more practical) can be used to represent all possible permutations. We call this table t and its i th element t_i . Each element can be notated by:

$$t_i = t_i[3]t_i[2]t_i[1]t_i[0], \text{ where } \{t_i[3], t_i[2], t_i[1], t_i[0]\} = \{0, 1, 2, 3\}.$$

The hexadecimal representation of all 24 elements is given in Table 2. Now π_K

Table 2: Hexadecimal representation of the table for the keyed permutations.

i	t_i	i	t_i	i	t_i
1	0x78	9	0x87	17	0x6c
2	0x36	10	0x63	18	0xd8
3	0x2d	11	0xd2	19	0xe1
4	0xc9	12	0x39	20	0xc6
5	0x93	13	0x8d	21	0x27
6	0x72	14	0x1e	22	0xb1
7	0x4b	15	0xe4	23	0x4e
8	0x9c	16	0xb4	24	0x1b

can be defined as a permutation that consists of 5 stages. Each stage depends on the value of one of the five key bytes of $K = k_4 \dots k_0$. The first stage is the rotation of the rows:

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\pi_{k_0}^1} \begin{pmatrix} x_{t_{k_0}[0]3} & x_{t_{k_0}[0]2} & x_{t_{k_0}[0]1} & x_{t_{k_0}[0]0} \\ x_{t_{k_0}[1]3} & x_{t_{k_0}[1]2} & x_{t_{k_0}[1]1} & x_{t_{k_0}[1]0} \\ x_{t_{k_0}[2]3} & x_{t_{k_0}[2]2} & x_{t_{k_0}[2]1} & x_{t_{k_0}[2]0} \\ x_{t_{k_0}[3]3} & x_{t_{k_0}[3]2} & x_{t_{k_0}[3]1} & x_{t_{k_0}[3]0} \end{pmatrix}. \quad (19)$$

In the following four stages each column is permuted. For example, column 0 is permuted according to the value of k_1 :

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\pi_K} \begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{t_{k_0}[1]0} \\ x_{13} & x_{12} & x_{11} & x_{t_{k_0}[1]0} \\ x_{23} & x_{22} & x_{21} & x_{t_{k_0}[1]0} \\ x_{33} & x_{32} & x_{31} & x_{t_{k_0}[1]0} \end{pmatrix} \quad (20)$$

With this change we have the cipher

$$E_{K^0||K^1} = \sigma_{K_9^0} \circ \pi_{K_9^1} \circ \gamma \circ \prod_{i=8}^0 (\sigma_{K_i^0} \circ \theta \circ \pi_{K_i^1} \circ \gamma). \quad (21)$$

2.3.5 Keyed Byte-Wise Rotations

Another change that does not violate the security motivations is the addition of a (keyed) transformation that operates on each byte and does not compromise the properties of the S-box. Since it operates only on bytes it does not interfere with property 2.2 and 2.3. To keep property 2.1 intact a sufficient condition is that it only permutes bits. We propose to insert the keyed transformation β_K that rotates each byte of a state with an amount that depends on the value of K . There are $8^{16} = 2^{48}$ possible values for K . We propose to insert β_K between π and γ in (7) and (8).

The rotations are determined by the value of the 48-bit key K as follows. Let $X = x_{15}||\dots||x_0$ denote a 16 byte state and $K[x+l..x]$ the radix-2 value formed by the bits $x+l$ to x of K , then β_K is defined as

$$\beta_K(X) = (x_{15} \ll K[47..45])||\dots||(x_0 \ll K[2..0]). \quad (22)$$

With this change we have the cipher

$$E_{K^0||K^1||K^2} = \sigma_{K_9^0} \circ \beta_{K_9^2} \circ \pi_{K_9^1} \circ \gamma \circ \prod_{i=8}^0 (\sigma_{K_i^0} \circ \beta_{K_i^2} \circ \theta \circ \pi_{K_i^1} \circ \gamma). \quad (23)$$

Note that for reasons of efficiency it also might have been possible to perform key-dependent rotations only on a subset of the 16 bytes.

The change of the diffusion layer also implies that the main concerns of Murphy and Robshaw as described in [22] are resolved. They described a slightly modified version of this layer for which sixteen subsequent applications are equivalent to the identity transformation. Since for our design the diffusion layer depends on the key (of each round) such simplification would be key-dependable and only applicable for a negligible amount of keys.

2.3.6 Shannon's Construction

As a (final) addition to the cipher we propose an XDES like 'simple' key mixing as inner and outer layer. This does not interfere with any of the security motivations of Rijndael. We do not use the XDES construction of xoring a key of block length before and after the Rijndael-like cipher for the following reasons:

- Rijndael already mixes the key in the cipher with xors. As motivated in Section 2.2 it is preferable to have diverse methods of key mixing.
- Just xoring keys would give opportunity to mount advanced slide attacks as described by Biryukov and Wagner in [6]. See also Section 5.3.

Instead of that we choose in the outer rounds to mix a 128-bit key byte-wise using addition modulo 2^8 . We denote it with ψ_K :

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\psi_K} \begin{pmatrix} x_{03} + k_{03} & x_{02} + k_{02} & x_{01} + k_{01} & x_{00} + k_{00} \\ x_{13} + k_{13} & x_{12} + k_{12} & x_{11} + k_{11} & x_{10} + k_{10} \\ x_{23} + k_{23} & x_{22} + k_{22} & x_{21} + k_{21} & x_{20} + k_{20} \\ x_{33} + k_{33} & x_{32} + k_{32} & x_{31} + k_{31} & x_{30} + k_{30} \end{pmatrix}, \quad (24)$$

where $+$ denotes addition modulo 2^8 .

Because this mixing is byte-oriented reduced versions of the remaining cipher would be equally vulnerable as Rijndael to Square-like attacks, that are described in [1, 2, 11, 12, 19]. These kind of attacks exploit the slow diffusion of a cipher. In our construction these attacks can be made infeasible by adding a diffusion layer between the outer round key mixing and the first transformation of the Rijndael structure γ .

A similar approach was used in the design of AES-candidate MARS. We propose the following transformation $\nu : \{0,1\}^{128} \rightarrow \{0,1\}^{128}$. Let the input of ν be denoted by $x_{15} \dots x_0$, where each x_i is a byte. Then the output $x_{15} \dots x_0$ is derived iteratively by:

1. For $i = 0$ to 15 do
 $x_{i+1 \bmod 16} = x_{i+1 \bmod 16} \oplus S(x_i)$.
2. For $i = 0$ to 15 do
For $j = 0$ to 15, $j \neq i$, do
 $x_j = x_j \oplus S(x_i)$.

The inverse transformation ν^{-1} is given by:

1. For $i = 15$ to 0 do
For $j = 0$ to 15, $j \neq i$, do
 $x_j = x_j \oplus S(x_i)$.
 $x_{i+1 \bmod 16} = x_{i+1 \bmod 16} \oplus S(x_i)$.
2. For $i = 15$ to 0 do
 $x_{i+1 \bmod 16} = x_{i+1 \bmod 16} \oplus S(x_i)$.

This transformation has the following properties.

- Each byte of the output depends on the value of every input byte and every output byte. Hence, one could say it provides good diffusion and a fast avalanche of key bits. This makes the application of outer round

tricks more difficult, since they would involve guessing of entire 128-bit outer round key.

- Changing one input byte causes a change of all output bytes with high probability. This is caused by the previous item and the involved use of S-boxes and xors. It increases resistance against the dedicated Square attack and differential attacks.
- Due to the extensive use of the Rijndael S-box there are no predictable changes of small inputs with high probability or low probability. This increases resistance against attacks using truncated differentials.

2.3.7 The Cipher

After this change we have the final cipher

$$E_K = \psi_{K_1^3} \circ \nu^{-1} \circ \sigma_{K_{10}^0} \circ \beta_{K_9^2} \circ \pi_{K_9^1} \circ \gamma \circ \sigma_{K_9^0} \circ \prod_{i=8}^0 (\beta_{K_i^2} \circ \theta \circ \pi_{K_i^1} \circ \gamma \circ \sigma_{K_i^0}) \circ \nu \circ \psi_{K_0^3}. \quad (25)$$

Note that, contrary to Rijndael, it is not possible to have an equivalent inverse cipher structure.

2.3.8 The One-Way Functions

The cipher requires 4 128-bit keys to derive the subkeys for the transformations ψ , σ , π and β . We denote these keys with K^0 , K^1 , K^2 and K^3 for respectively ψ , σ , π and β , hence in the order of which they are applied in the encryption. These keys should be indistinguishable from independent keys as motivated in Section 2.1. To derive the subcipher keys from the user key, when the key length is smaller than 512 bits, we propose to use the following simple construction.

$$K^i = \text{trunc}_{128}(F(K, i)) \text{ for } i = 0, \dots, 3, \quad (26)$$

where F is a suitable one-way function trunc_{128} truncates the result to 128 bits, by taking the left 128 bits, if necessary. For F we propose two possibilities.

A dedicated one-way function It is very reasonable to choose a dedicated one-way function f in the construction of F :

$$F(K, i) = f(K || i), \text{ for } i = 0, \dots, 3 \quad (27)$$

where f can be chosen from MD5, SHA-1, RIPEMD-160, or other proposals. Reasons to do this is that for many applications also a dedicated hash function is implemented.

The cipher in one-way function mode To save on memory by implementing an extra one-way function and to have a one-way function based on the same security principles as the block cipher one can use the cipher itself in a one-way function mode. We propose the use of a one-way function mode inspired by the Matyas-Meyer-Oseas hash function construction, as described in [21]. First an alternative method has to be described to specify the subcipher keys. If one uses the cipher with a 512-bit key, one can derive the 4 subcipher keys by taking the subsequent 128 bits of the user key. Hence, we propose the use of a 512-bit key that depends on i . Define the 8-byte constant

$$c = 0x0123456789abcdef.$$

Note that c is a string of 128 bits. Now we define the function F as

$$F(K, i) = E_{(c|c|c|c)}(K \ll 4 * i) \oplus (K \ll 4 * i) \quad (28)$$

in the case that K has 128 bits. This function has the properties of the Matyas-Meyer-Oseas construction for a given IV .

For the case that the user key K is not a multiple of 128-bit values padding is necessary. We propose to concatenate zeroes to the left of K until $0 \dots 0K = \tilde{K}_0 \dots \tilde{K}_{l-1}$ has a length of $l \cdot 128$ bits and every $\tilde{K}$ has a length of 128 bits. Then K_i can be computed according to the Matyas-Meyer-Oseas construction.

$$F_0 = E_{(c|c|c|c)}(\tilde{K}_0 \ll 4 * i) \quad (29)$$

$$F_{j+1} = E_{(c|c|c|c)}(\tilde{K}_j \ll 4 * i) \oplus (\tilde{K}_j \ll 4 * i) \text{ for } j = 0 \dots l - 2. \quad (30)$$

$$F(K, i) = F_{l-1} \quad (31)$$

2.3.9 Round Key Derivation

For the subciphers defined in Section 2.3.4 and 2.3.5 we need to define a key schedule to derive their round keys from the K^i 's, that were derived with the one-way function. We propose the following. Use the key schedule of Rijndael for all subciphers. Rijndael is analyzed with this key schedule and hence can be trusted. The choice of key scheduling for the other subciphers is less important from a security point of view since the changes to the cipher increase the diffusion and confusion [24] of the structure, such that attacks that exploit properties of the key schedule are less likely.

We use the notation for f_{rk} as defined in (18). For each keyed transformation the roundkeys are derived as follows.

- ψ needs two 128-bit subkeys K_0^3 and K_1^3 . They are derived from K^3 as

$$K_0^3, K_1^3 = f_{rk}(K^3, 0), f_{rk}(K^3, 1).$$

- σ is the original Rijndael key xoring and needs eleven 128-bit subkeys. These keys are derived from K^0 as

$$K_0^0, \dots, K_{10}^0 = f_{rk}(K^1, 0), \dots, f_{rk}(K^1, 10).$$

- A round key for π can be represented by 5 bytes with a value smaller than 24. A 16 byte round key $k_{15} \dots k_0$ is derived from K^1 for each round. From this key the first 5 bytes that are smaller than 24 are taken as subkey bytes for π . If $5 - s$ key bytes are smaller than 24, then the values of the first s bytes modulo 16 are chosen:
 1. For $i = 0$ to 9 do
 - a. Compute $k_{15} \dots k_0 = f_{\text{rk}}(K^1, i)$
 - b. Set counter $c = 0$.
 - c. For $j = 0$ to 15 do
 - If $k_j < 24$ then $K_i^1[c] = k_j, c++$.
 - d. If $c < 5$
 - For $j = c$ to 5 do
 - $K_i^1[c] = k_{j-c} \bmod 16$.
- β needs per round sixteen 3-bit values. These 3-bit values are the three least significant bits of each key byte of a round key that can be derived from K^3 . Hence, for $i = 0, \dots, 9$

$$k_{15} \dots k_0 = f_{\text{rk}}(K^2, i) \quad (32)$$

$$K_i^2 = \text{LSB}_3(k_{15}) \dots \text{LSB}_3(k_0). \quad (33)$$

3 Specification

This section contains a complete description of GRAND CRU and in this sense summarizes Sect 2.

3.1 Notation

A transformation $\lambda : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}$ can be denoted as:

$$\lambda(X) = Y, \quad (34)$$

and expressing X and Y as 16-byte strings:

$$\lambda(x_{33}x_{32}x_{31}x_{30}x_{23} \dots x_{01}x_{00}) = y_{33}y_{32}y_{31}y_{30}y_{23} \dots y_{01}x_{00}, \quad (35)$$

where all x_{ij} and y_{ij} are byte values. The transformations of GRAND CRU also can be viewed upon as operating on a 4×4 byte array or state. In our notation (34) and (35) are equivalent to

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\lambda} \begin{pmatrix} y_{03} & y_{02} & y_{01} & y_{00} \\ y_{13} & y_{12} & y_{11} & y_{10} \\ y_{23} & y_{22} & y_{21} & y_{20} \\ y_{33} & y_{32} & y_{31} & y_{30} \end{pmatrix}. \quad (36)$$

Furthermore an extra useful notation is to express a transformation λ as a transformation on the separate columns:

$$(x_3|x_2|x_1|x_0) \xrightarrow{\lambda} (y_3|y_2|y_1|y_0). \quad (37)$$

Similar as for the cipher states 128-bit subkeys can also be represented by a 4×4 -array:

$$k_{33}k_{32}k_{31}k_{30}k_{23} \dots k_{01}k_{00} = \begin{pmatrix} k_{03} & k_{02} & k_{01} & k_{00} \\ k_{13} & k_{12} & k_{11} & k_{10} \\ k_{23} & k_{22} & k_{21} & k_{20} \\ k_{33} & k_{32} & k_{31} & k_{30} \end{pmatrix} = (k_3|k_2|k_1|k_0). \quad (38)$$

3.2 Transformations

The building blocks of GRAND CRU for encryption and decryption are 4 keyed and 3 non-keyed invertible transformations, that operate on 128-bit strings. They are defined as follows.

Round Key Addition σ This is performed by xoring the input with a 128-bit key K :

$$\sigma(A)_K = A \oplus K, \text{ for } K, A \in \{0, 1, \}^{128}. \quad (39)$$

The inverse of σ_K is σ_K .

Non-linear Substitution Transformation γ This uses an 8×8 S-box S , which operates on each of the bytes of the state independently:

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\gamma} \begin{pmatrix} S(x_{03}) & S(x_{02}) & S(x_{01}) & S(x_{00}) \\ S(x_{13}) & S(x_{12}) & S(x_{11}) & S(x_{10}) \\ S(x_{23}) & S(x_{22}) & S(x_{21}) & S(x_{20}) \\ S(x_{33}) & S(x_{32}) & S(x_{31}) & S(x_{30}) \end{pmatrix}. \quad (40)$$

The S-box is given in Appendix A. The inverse of γ the S-box should be replaced with its inverse in 40.

Mix-Column Transformation θ The transformation θ can be represented as a matrix multiplication modulo 2^8 :

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\theta} \begin{pmatrix} 0x02 & 0x03 & 0x01 & 0x01 \\ 0x01 & 0x02 & 0x03 & 0x01 \\ 0x01 & 0x01 & 0x02 & 0x03 \\ 0x03 & 0x01 & 0x01 & 0x02 \end{pmatrix} \cdot \begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix}. \quad (41)$$

Its inverse is given by

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\theta^{-1}} \begin{pmatrix} 0x0e & 0x0b & 0x0d & 0x09 \\ 0x09 & 0x0e & 0x0b & 0x0d \\ 0x0d & 0x09 & 0x0e & 0x0b \\ 0x0b & 0x0d & 0x09 & 0x0e \end{pmatrix} \cdot \begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix}. \quad (42)$$

Keyed Permutation π This permutation can be implemented by representing each of the five separate permutations on four bytes or rows by 5 key bits that represent a value smaller than 24. A table of 8-bit 24 entries is used to represent all possible permutations. We call this table t and its i th element t_i . Each element can be notated by:

$$t_i = t_i[3]t_i[2]t_i[1]t_i[0], \text{ where } \{t_i[3], t_i[2], t_i[1], t_i[0]\} = \{0, 1, 2, 3\},$$

where each $t_i[j]$ is represented by two bits. The hexadecimal representation of all 24 elements is given in Table 2.

π_K can be defined as a permutation that consists of 5 stages. Each stage depends on the value of one of the five key bytes of $K = k_4 \dots k_0$. The first stage is the rotation of the rows:

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\pi_{k_0}^1} \begin{pmatrix} x_{t_{k_0}[0]3} & x_{t_{k_0}[0]2} & x_{t_{k_0}[0]1} & x_{t_{k_0}[0]0} \\ x_{t_{k_0}[1]0} & x_{t_{k_0}[1]3} & x_{t_{k_0}[1]2} & x_{t_{k_0}[1]1} \\ x_{t_{k_0}[2]1} & x_{t_{k_0}[2]0} & x_{t_{k_0}[2]3} & x_{t_{k_0}[2]2} \\ x_{t_{k_0}[3]2} & x_{t_{k_0}[3]1} & x_{t_{k_0}[3]0} & x_{t_{k_0}[3]3} \end{pmatrix}. \quad (43)$$

In the following four stages each column is permuted. For example, column 0 is permuted according to the value of k_1 :

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\pi_K} \begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{t_{k_0}[1]0} \\ x_{13} & x_{12} & x_{11} & x_{t_{k_0}[1]0} \\ x_{23} & x_{22} & x_{21} & x_{t_{k_0}[1]0} \\ x_{33} & x_{32} & x_{31} & x_{t_{k_0}[1]0} \end{pmatrix} \quad (44)$$

Keyed Byte-Wise Rotation β The transformation β is keyed by a 48-bit key K . Let $X = x_{15} || \dots || x_0$ denote a 16 byte state and $K[x+l..x]$ the radix-2 value formed by the bits $x+l$ to x of K , then β_K is defined as

$$\beta_K(X) = (x_{15} \ll K[47..45]) || \dots || (x_0 \ll K[2..0]). \quad (45)$$

The inverse of β is given by

$$\beta_K^{-1}(X) = (x_{15} \gg K[47..45]) || \dots || (x_0 \gg K[2..0]). \quad (46)$$

Outer Round Key Addition ψ_K This transformation is a byte-wise addition of a key $K = k_{33}k_{32}k_{31}k_{30}k_{23} \dots k_{01}k_{00}$ with its input:

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\psi_K} \begin{pmatrix} x_{03} + k_{03} & x_{02} + k_{02} & x_{01} + k_{01} & x_{00} + k_{00} \\ x_{13} + k_{13} & x_{12} + k_{12} & x_{11} + k_{11} & x_{10} + k_{10} \\ x_{23} + k_{23} & x_{22} + k_{22} & x_{21} + k_{21} & x_{20} + k_{20} \\ x_{33} + k_{33} & x_{32} + k_{32} & x_{31} + k_{31} & x_{30} + k_{30} \end{pmatrix}, \quad (47)$$

where $+$ denotes addition modulo 2^8 . Its inverse is

$$\begin{pmatrix} x_{03} & x_{02} & x_{01} & x_{00} \\ x_{13} & x_{12} & x_{11} & x_{10} \\ x_{23} & x_{22} & x_{21} & x_{20} \\ x_{33} & x_{32} & x_{31} & x_{30} \end{pmatrix} \xrightarrow{\psi_K^{-1}} \begin{pmatrix} x_{03} - k_{03} & x_{02} - k_{02} & x_{01} - k_{01} & x_{00} - k_{00} \\ x_{13} - k_{13} & x_{12} - k_{12} & x_{11} - k_{11} & x_{10} - k_{10} \\ x_{23} - k_{23} & x_{22} - k_{22} & x_{21} - k_{21} & x_{20} - k_{20} \\ x_{33} - k_{33} & x_{32} - k_{32} & x_{31} - k_{31} & x_{30} - k_{30} \end{pmatrix}, \quad (48)$$

where $-$ denotes subtraction modulo 2^8 .

Unkeyed Diffusion Layer ν Let the input of ν be denoted by $x_{15} \dots x_0$, where each x_i is a byte. Then the output $x_{15} \dots x_0$ is derived iteratively by:

1. For $i = 0$ to 15 do
 $x_{i+1 \bmod 16} = x_{i+1 \bmod 16} \oplus S(x_i).$
2. For $i = 0$ to 15 do
 For $j = 0$ to 15, $j \neq i$, do
 $x_j = x_j \oplus S(x_i).$

The inverse transformation ν^{-1} is given by:

1. For $i = 15$ to 0 do
 For $j = 0$ to 15 , $j \neq i$, do
 $x_j = x_j \oplus S(x_i)$.
 $x_{i+1 \bmod 16} = x_{i+1 \bmod 16} \oplus S(x_i)$.
2. For $i = 15$ to 0 do
 $x_{i+1 \bmod 16} = x_{i+1 \bmod 16} \oplus S(x_i)$.

3.3 The Cipher

Encryption is defined by:

$$E_K = \psi_{K_1^3} \circ \nu^{-1} \circ \sigma_{K_{10}^0} \circ \beta_{K_9^2} \circ \pi_{K_9^1} \circ \gamma \circ \sigma_{K_9^0} \circ \prod_{i=8}^0 (\beta_{K_i^2} \circ \theta \circ \pi_{K_i^1} \circ \gamma \circ \sigma_{K_i^0}) \circ \nu \circ \psi_{K_0^3}, \quad (49)$$

where subkeys K_j^i are defined in the next section.

Decryption is defined by:

$$E_K^{-1} = \psi_{K_0^3}^{-1} \circ \nu^{-1} \circ \prod_{i=0}^8 (\sigma_{K_i^0} \circ \gamma^{-1} \circ \pi_{K_i^1}^{-1} \circ \theta^{-1} \circ \beta_{K_i^2}^{-1}) \circ \sigma_{K_9^0} \circ \gamma^{-1} \circ \pi_{K_9^1}^{-1} \circ \beta_{K_9^2}^{-1} \circ \sigma_{K_{10}^0} \circ \nu \circ \psi_{K_1^3}^{-1}. \quad (50)$$

3.4 Key Schedule

The key schedule of GRAND CRU is performed in two stages. First four 128-bit keys are derived from the userkey. From each of those keys the round keys for the keyed transformations will be derived.

Building Blocks The key schedule makes use of two transformations Rotl and Γ . We define the transformation Rotl, which operates on columns of a key state. It rotates the bytes of a column as follows.

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \xrightarrow{\text{Rotl}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_0 \end{pmatrix} \quad (51)$$

Furthermore the transformation Γ also operates on the bytes of a column and is defined as:

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \xrightarrow{\Gamma} \begin{pmatrix} S(x_0) \\ S(x_1) \\ S(x_2) \\ S(x_3) \end{pmatrix}. \quad (52)$$

From a 128-bit key K a set of $n + 1$ 128-bit keys

$$K_n \dots K_1, K_0 = (K'_{4n+3}|K'_{4n+2}|K'_{4n+1}|K'_{4n}), \dots, (K'_7|K'_6|K'_5|K'_4), (K'_3|K'_2|K'_1|K'_0), \quad (53)$$

where each K' is a column of 4 bytes, can be derived as follows:

$$K_0 = K^u \quad (54)$$

and for $i > 3$:

$$K'_i = K'_{i-4} \oplus \Gamma(\text{Rotl}(K'_{i-1}) \oplus c(i/4), \text{ if } 4|i, \quad (55)$$

$$K'_i = K'_{i-4} \oplus K'_{i-1} \text{ else,} \quad (56)$$

where $c(i)$ are the round constants of the key schedule as defined in Table 1.

A shortened notation to denote the derivation of K_i from K is

$$K_i = f_{\text{rk}}(K, i). \quad (57)$$

Round Key Derivation For each keyed transformation the roundkeys are derived as follows.

- ψ needs two 128-bit subkeys K_0^3 and K_1^3 . They are derived from K^3 as

$$K_0^3, K_1^3 = f_{\text{rk}}(K^3, 0), f_{\text{rk}}(K^3, 1).$$

- σ is the original Rijndael key xoring and needs eleven 128-bit subkeys. These keys are derived from K^0 as

$$K_0^0, \dots, K_{10}^0 = f_{\text{rk}}(K^1, 0), \dots, f_{\text{rk}}(K^1, 10).$$

- A round key for π can be represented by 5 bytes with a value smaller than 24. A 16 byte round key $k_{15} \dots k_0$ is derived from K^1 for each round. From this key the first 5 bytes that are smaller than 24 are taken as subkey bytes for π . If $5 - s$ key bytes are smaller than 24, then the values of the first s bytes modulo 16 are chosen:

1. For $i = 0$ to 9 do
 - a. Compute $k_{15} \dots k_0 = f_{\text{rk}}(K^1, i)$
 - b. Set counter $c = 0$.
 - c. For $j = 0$ to 15 do
 - If $k_j < 24$ then $K_i^1[c] = k_j, c++$.
 - d. If $c < 5$
 - For $j = c$ to 5 do
 - $K_i^1[c] = k_{j-c} \bmod 16$.

- β needs per round sixteen 3-bit values. These 3-bit values are the three least significant bits of each key byte of a round key that can be derived from K^3 . Hence, for $i = 0, \dots, 9$

$$k_{15} \dots k_0 = f_{\text{rk}}(K^2, i) \quad (58)$$

$$K_i^2 = \text{LSB}_3(k_{15}) \dots \text{LSB}_3(k_0). \quad (59)$$

Derivation Subcipher Keys The cipher requires 4 128-bit keys to derive the subkeys for the transformations ψ , σ , π and β from the 128-bit user key K^u . We denote these keys with K^0 , K^1 , K^2 and K^3 for respectively ψ , σ , π and β , hence in the order of which they are applied in the encryption.

Define the 8-byte constant

$$c = 0x0123456789abcdef.$$

Now we define

$$K^i = E_{(c|c|c|c)}(K^u \ll 4 * i) \oplus (K^u \ll 4 * i), \quad (60)$$

for $i = 0, \dots, 3$.

4 Implementation

One of the most important motivations in our design was resistance against side-channel attacks. Most suitable targets for these kind of attacks are smart cards with 8-bit processors. On these kind of processors the cipher would have a lower performance than Rijndael, due to the extra operations. Based on preliminary experiments we estimate that the efficiency of an GRAND CRU encryption is approximately three times as slow as a Rijndael encryption. We believe that this performance drop is compensated by the added security in the proposed cipher.

Note that due to the increased amount of subkeys the memory requirements of an implementation increase. However, since our design uses the Rijndael key schedule it also benefits from its feature of on-the-fly key computation, as given in property 2.4. Without on-the-fly key computation for our proposal 402 bytes of round key material need to be stored. However, using property 2.4 only 512 bits of memory are needed for subkey storage.

On other platforms due to the insertion of many key dependent operations the cipher Rijndael will have a much better performance than our design. Moreover, the mathematical optimization which caused Rijndael to have a very good performance on 32-bit and 64-bit processors is not possible anymore. Furthermore, in hardware extra resources will be needed to implement all the extra operations. Possibly another application of the design strategy, beginning with Rijndael could result in a cipher with better performance. It still is an open question if efficient design over many platforms is possible with this strategy.

4.1 Efficiency

Currently we are working on an implementation of GRAND CRU for the Motorola 6805 8-bit processor architecture. Based on the current status we present estimates for a complete implementation. Estimates are made for the efficiency of encryption, decryption and key setup. Besides efficiency also estimates for memory requirements are given, since these are relevant for low-end smart cards with memory restrictions. The estimates are resumed in Table 3.

Table 3: Efficiency and memory estimates of an implementation of GRAND CRU on Motorola 6805 8-bit processors.

	Encryption	Decryption	Key setup
Time (cycles/byte)	3750	3750	187500
Time (cycles/block)	60000	60000	300000
ROM (bytes)	2200	2200	100
RAM (bytes)	120	120	120

Furthermore, we have tested the efficiency of the reference ANSI C implementation on an Intel Pentium 200 MHz, running Linux 2.0.38, and on an Intel

XEON 550 MHz, running Linux 2.2.14. The implementation was compiled with gcc, with flags -O and -O2. Efficiency of encryption, decryption and key setup were tested. These results are given in Table 4. These figures correspond with approximate speeds of 1.5 Mbit/s for the XEON and 0.5 Mbit/s for the Pentium. Optimized C-code is not available yet.

Table 4: Efficiency results of reference ANSI C implementation on an Intel Pentium 200 MHz, running Linux 2.0.38, and on an Intel XEON 550 MHz, running Linux 2.2.14.

	Encryption (cycles/block)	Decryption (cycles/block)	Key setup (cycles/setup)
Pentium	45000	65000	200000
XEON	65000	90000	300000

4.2 Avoiding Implementation Weaknesses

In recent years several implementation based or side channel attacks were introduced. The most prominent are the following two:

- **Timing attacks** were introduced in the open literature by Kocher in [17]. These attacks exploit the fact that the execution time of some implementations of cryptographic functions depends on the key material.
- **Power attacks** are described by Kocher *et al.* in [18]. the idea is to extract the key of a cryptographic device by analyzing its power consumption during cryptographic computations.

Weaknesses to timing attacks are mostly caused by optimized implementations. When implementing GRAND CRU, one has to take into account that the execution time of each operation does not vary according to the input values. Special attention has to be given to the implementation of the data dependent rotations, since these have showed to be vulnerable to timing analysis (see for example [14]).

In general power attacks are more difficult to prevent. As was the case for timing attacks, here operations that have as input key material need to be implemented such that the power consumption does not depend on the input (or at the least the dependence can not be exploited). There are several techniques, for which the relevance and effectiveness also depend on the platform. Depending on the platform different operations are better suitable for secure implementation. The fact that the security of GRAND CRU is provided by different key mixing operations is an advantage in the sense that only one key mixing operation has to be securely implemented. Although we would recommend if possible to implement all key mixing operations in a secure way.

Techniques that can be used for a power attack resistant implementation of GRAND CRU include:

- Randomization of the order of operations. Due to the byte oriented structure of GRAND CRU, the algorithm is particularly suited for this approach.
- Implementation of key dependent instructions such that the key material is not directly combined with known values. Examples include k -way split [8] or the duplication method [13]. These methods are applicable to the key xoring of σ and key addition of ψ .

Furthermore, additional general techniques based on hardware can be used like increasing noise in power consumption, adding on-chip power supply, desynchronization of instructions, etc.

5 Security Analysis

In the design rationale we have motivated each design decision to such a degree that it is extremely unlikely that the cipher contains any inserted and/or hidden weaknesses.

In Section 2.3 we have shown that the insertion and adding of operations did not decrease the security of the Rijndael-like subcipher. Rijndael also is secure if none of the other subciphers is added. We make the following conjecture

Conjecture 5.1 *Each method that would successfully break our proposal would also break Rijndael.*

This would certainly hold if all the subkeys for π and β were chosen such that Rijndael would be the core in between the outer round permutations. The only case in which conjecture 5.1 would not hold would be in the case of the existence of weak keys. The existence of such weak keys would be very unlikely. Besides that, it is not even clear which weakness or special property these keys should induce. However, since we have no proof statement 5.1 is posed as a conjecture.

Based on this we consider the complete cipher to have sufficient resistance against any generic brute force attacks or any state-of-the-art short-cut attacks.

One could also extract each subcipher in a similar way from the structure. However, *we certainly do not claim that any subcipher that is extracted from the structure in such a way is a secure block cipher*. For example: the cipher defined by the two outer key mixings needs a good diffusion layer in between. Otherwise, it could be ‘easily’ broken by several techniques. Hence we analyze the security of the other subciphers in a structure, where all but one subcipher key are known or chosen. Note that if the outer round keys are known, these outer rounds could as well be omitted. Hence, if these keys are known the outer round does not contribute to the security of any other subcipher. We omit this transformations in the analysis models in Section 5.1 and 5.2.

5.1 The Permutation Cipher

The cipher we analyze can be modeled as:

$$E_{K^1} = \pi_{K_9^1} \circ \gamma'_9 \circ \prod_{i=8}^0 (\theta \circ \pi_{K_i^1} \circ \gamma'_i), \quad (61)$$

where γ'_i is a transformation that performs a known S-box transformation on each byte of a state, but the S-boxes can (and, if this is no chosen subcipher key attack, with high probability will) be different for each pair of byte position and value of i .

The keyed permutation as described in Section 2.3.4 is determined by just 22 key bits per round. This is very little for a cipher that only consists of 10 rounds. It is possible to mount a basic meet-in-the-middle attack [21] that needs 2^{110} half encryptions/decryptions and storage for 2^{110} plaintext/‘ciphertext’-pairs. This is more efficient than exhaustive key search. Time-memory tradeoffs are

possible. However, it is not very likely that this attack could be modified to be feasible in practice.

Furthermore, due to the diffusion and confusion provided by the cipher structure we have found no method to exploit the seemingly simple key dependent permutations. Hence, we see no reason to judge this subcipher as being superfluous or to increase the number of rounds.

5.2 The Rotation Cipher

The cipher we analyze can be modeled as:

$$E_{K^2} = \beta_{K_9^3} \circ \pi'_9 \circ \prod_{i=8}^1 (\gamma'_i \circ \beta_{K_i^3} \circ \theta \circ \pi'_i) \circ \sigma'_0 \circ \beta_{K_0^3}, \quad (62)$$

where π'_i is a known byte permutation for each i , γ'_i is a transformation that performs a known S-box transformation on each byte of a state, and σ'_0 is an xor with a known constant. Note that γ'_i still has optimal properties for LC and DC resistance and that the diffusion properties due to the $\theta \circ \pi'_i$ -transformations still hold.

Its security is based on key-dependent rotations. Block ciphers for which the security is (heavily) based on data-dependent rotations are RC5 and its successor and AES-candidate RC6. Due to the extensive analysis of this ciphers in [1, 5, 7, 16] much is known about methods to exploit data-dependent rotations. Due to the diffusion and confusion properties of the structure the cipher seems to be sufficiently secure.

5.3 The Outer Round Cipher

Here the model is:

$$E_{K^3} = \psi_{K_1^3} \circ \chi \circ \psi_{K_0^3}, \quad (63)$$

where χ is a known transformation with very good diffusion and confusion.

In [6] advanced slide attacks were introduced to attack XDES. The fact that χ is not keyed makes such an attack in this case even more efficient. This attack is illustrated as follows. Suppose that in (63) the ψ transformation would xor its input with a key. If an attacker obtains $2^{64.5}$ plaintext/ciphertext pairs (P, C) , then with high probability for one pair (P, C) , (P', C') of those holds

$$P \oplus K_0^3 = \chi^{-1}(C') \text{ and } P' \oplus K_0^3 = \chi^{-1}(C). \quad (64)$$

From this follows that $C \oplus C' = K_1^3$ and hence one can easily deduce K_0^3 . From (64) also follows that $P \oplus P' = E^{-1}(C) \oplus E^{-1}(C')$. Hence, if the attacker finds such a pair, with high probability he can compute K_1^3 and K_0^3 .

Because ψ does its key mixing with additions modulo 2^8 this kind of attack is not applicable.

6 Advantages and Limitations

In this section we resume the main advantages and limitations of the design strategy of multiple layered security. The limitations are mainly caused by the restrictions that seem to be inherent to the design strategy.

Advantages

- The design has a good performance on low-end smart cards.
- The security is based on several different complementary security motivations.
- Part of the security is based on a well evaluated cipher: Rijndael. Rijndaels resistance against known short cut attacks
- The use of different operations to mix in the keys implies that an attacker would need different techniques.
- Enhanced security against implementation based attacks.
- Enhanced security against developments in cryptanalytic attack methods.
- Better performance is possible compared to ciphers that offer a similar level of security, i.e., compared to sequences of ciphers that are designed on different design principles.
- Code for the cipher can be written on basis of known and available code of the well-known and widely implemented cipher Rijndael.
- The result of the design strategy might seem to lack simplicity and elegance, but this can be easily be resolved by taking also notice of the design process.

Limitations

- A design that is based on one motivation would probably have better encryption/decryption speed and key-setup time.
- With this strategy it seems difficult to design a cipher that is very efficient over a large variety of platforms.
- It is inherent to the strategy that a large amount of memory is required for subkeys. For example, for our design 512 bits of subkey material would need to be stored during encryption or decryption, even with though it uses the on-the-fly key schedule of Rijndael.
- The use of different key mixing operations increases the cost for hardware implementation and makes optimizing code less trivial.

7 Miscellaneous

7.1 Intellectual Property

As far to our knowledge the algorithm there are no patents or patent applications covering the use of the algorithm `GRAND CRU`.

Acknowledgements

Of course, we would like to thank Joan Daemen and Vincent Rijmen for designing Rijndael, as well as for giving interesting feedback. Furthermore, we acknowledge Bart Preneel for interesting discussions. We also thank Jorge Nakahara for designing a preliminary version of the diffusion transformation. The reference ANSI C code that is submitted to NESSIE is based on a reference implementation, written by Vincent Rijmen and Paulo Barreto.

References

- [1] National Institute For Standards and Technology (NIST), “AES, A Crypto Algorithm for the Twenty-first Century,” <http://www.nist.gov/aes>.
- [2] O. Baudron, H. Gilbert, L. Granboulan, H. Handschuh, A. Youx, P. Nguyen, F. Noilhan, D. Pointcheval, T. Pornin, G. Poupard, J. Stern, S. Vaudenay, “Report on the AES Candidates,” *Proc. of the Second AES Candidate Conference*, 1999, pp. 1–12.
- [3] E. Biham, A. Shamir, *Differential Cryptanalysis of the Data Encryption Standard*, Springer-Verlag, 1993.
- [4] E. Biham, A. Shamir, “Power Analysis of the AES Candidates,” *Proc. of the Second AES Candidate Conference*, 1999, pp. 115–121.
- [5] A. Biryukov, E. Kushilevitz, “Improved Cryptanalysis of RC5,” *Proc. Eurocrypt’98, LNCS 1403*, Springer-Verlag, 1998, pp. 85–99.
- [6] A. Biryukov, D. Wagner, “Advanced Slide Attacks,” *Advances in Cryptology, Proc. Eurocrypt’00*, to appear.
- [7] J. Borst, B. Preneel, J. Vandewalle, “Linear Cryptanalysis of RC5 and RC6,” *Fast Software Encryption, LNCS 1636*, Springer-Verlag, 1999, pp. 16–30.
- [8] S. Chari, C. Jutla, J. Rao, P. Rohatgi, “Towards Sound Approaches to Counteract Power-Analysis Attacks,” *Advances in Cryptology, Proc. Crypto’99, LNCS 1666*, M. Wiener, Ed., Springer-Verlag, 1999, pp. 398–412.
- [9] J. Daemen, “*Cipher and Hash Function Design*,” PhD. Thesis, Departement Elektrotechniek-ESAT/COSIC, Katholieke Universiteit Leuven, March 1995.
- [10] J. Daemen, V. Rijmen, *AES Proposal: Rijndael*, available via [1].
- [11] N. Ferguson, J. Kelsey, B. Schneier, M. Stay, D. Wagner, D. Whiting, “Improved Cryptanalysis of Rijndael,” *Fast Software Encryption 2000*, to appear.
- [12] H. Gilbert, M. Minier, “A Collision Attack on 7 Rounds of Rijndael,” *Proc. of the Third AES Candidate Conference*, 2000, pp. 230–241.
- [13] L. Goubin, J. Patarin, “*DES and Differential Power Analysis, The “Duplication” Method*,” *Proceedings of CHES’99, LNCS 1717*, C. Koc, C. Paar, Eds. (Springer-Verlag, 1999).
- [14] H. Handschuh, H.M. Heys, “A Timing Attack on RC5,” *Proc. Workshop on Selected Areas of Cryptography, SAC ’98, LNCS 1556*, Springer-Verlag, 1998.

- [15] J. Kilian, P. Rogaway, “How to protect DES against exhaustive key search,” *Proc. Crypto’96, LNCS 1109*, Springer-Verlag, 1996, pp. 252–267.
- [16] L. Knudsen, W. Meier, “Correlations in RC6,” Fast Software Encryption Workshop 2000, to appear.
- [17] P. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” *Advances in Cryptology, Proc. Crypto’96, LNCS 1109* N. Koblitz, Ed. (Springer-Verlag, 1996) 104–113.
- [18] P. Kocher, J. Jaffe, B. Jun, “Differential Power Analysis,” *Advances in Cryptology, Proc. Crypto’99, LNCS 1666* M. Wiener, Ed. (Springer-Verlag, 1999) 388–397.
- [19] S. Lucks, “Attacking Seven Rounds of Rijndael under 192-bit and 256-bit Keys,” *Proc. of the Third AES Candidate Conference*, 2000, pp. 215–229.
- [20] M. Matsui, “Linear cryptanalysis method for DES cipher,” *Proc. Eurocrypt’93, LNCS 765*, Springer-Verlag, 1994, pp. 386–397.
- [21] A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1997.
- [22] S. Murphy, M. Robshaw, “*New Observations on Rijndael*,” comment to NIST, August 2000.
- [23] *New European Schemes for Signatures, Integrity, and Encryption*, project within the Information Societies Technology (IST) Programme of the European Commission, <http://www.cryptonessie.com>.
- [24] C.E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, Vol. 28, Oct. 1949, pp. 656–715.
- [25] S. Vaudenay, “Provable Security for Block Ciphers by Decorrelation,” *STACS 98, LNCS 1373*, Springer-Verlag, 1998, pp. 249–275.

A S-Box

The S-box entries from $S[0]$ to $S[255]$:

99	124	119	123	242	107	111	197
48	1	103	43	254	215	171	118
202	130	201	125	250	89	71	240
173	212	162	175	156	164	114	192
183	253	147	38	54	63	247	204
52	165	229	241	113	216	49	21
4	199	35	195	24	150	5	154
7	18	128	226	235	39	178	117
9	131	44	26	27	110	90	160
82	59	214	179	41	227	47	132
83	209	0	237	32	252	177	91
106	203	190	57	74	76	88	207
208	239	170	251	67	77	51	133
69	249	2	127	80	60	159	168
81	163	64	143	146	157	56	245
188	182	218	33	16	255	243	210
205	12	19	236	95	151	68	23
196	167	126	61	100	93	25	115
96	129	79	220	34	42	144	136
70	238	184	20	222	94	11	219
224	50	58	10	73	6	36	92
194	211	172	98	145	149	228	121
231	200	55	109	141	213	78	169
108	86	244	234	101	122	174	8
186	120	37	46	28	166	180	198
232	221	116	31	75	189	139	138
112	62	181	102	72	3	246	14
97	53	87	185	134	193	29	158
225	248	152	17	105	217	142	148
155	30	135	233	206	85	40	223
140	161	137	13	191	230	66	104
65	153	45	15	176	84	187	22